to help our users ensure things don't fall into the cracks. For example, you've been mentioned in an issue or comment, or in this case had a failed pipeline.

This way it's easier for users to go about their day, with less repetition, and fewer mistakes.

So that's a quick review of what we would call the "Negative Path". Let's take a look at some of the available options for reviewing pipeline performance.

Monitoring CI (Pipeline History, CI Monitoring with Prometheus)

> Click on Pipelines tab for the list of all pipelines

First, GitLab retains a complete pipeline history for all current and old pipelines, stages, and jobs. We track the status of each job, but also retain the build logs permanently. As for the artifacts a default expiration can be set to manage space, but it is easily overridden from within the .gitlab-ci.yml file.

This page is great for checking out the status of pipelines for a variety of branches, and getting insight into what the CI system is doing.

But that isn't the only way to get an understanding of what the CI system is currently executing. We also offer a wealth of metrics that are tracked with the integrated Prometheus monitoring system. Let's take a look at the public monitoring dashboard for GitLab.com

> Show dashboards.gitlab.com

Here you can see a wide variety of metrics, ranging from the number of jobs in each state to how many parallel jobs a given runner is executing. All of these metrics are available not just on SaaS, but also self-managed as well.

Analytics (Charts, Static Analysis, Code Coverage, Cycle Analytics, APM)

GitLab also provides methods to understand the health of the CI pipelines for a particular job as well. We have a dedicated analytics page we call Charts, which shows additional information for each project.

> Open Pipelines > Charts

Here you can get insight into the average success rate and execution duration of your pipelines. For us, ours are executing quite quickly, usually under 2 minutes.

We also provide historical insight into how the success rate is changing over time, and the number of jobs executed within the last week, month and year.

And we've already taken a look at some of the other analytics services we offer as part of CI:

- Our static analysis intelligence with codeclimate
- Code coverage parsing
- and cycle analytics, for team health and efficiency.

GitLab Runner (Shared, Specific, Autoscaling)

Now I've mentioned our Runner a couple times, but we've been mostly looking at what it can do. Next we'll take a few minutes to talk about this important part of GitLab CI.

The GitLab Runner is a small portable app, which we build for a wide variety of platforms. It's essentially the worker bee, which picks up and executes the jobs we specify in our pipelines.

In our Pipelines settings page is where a developer can take a look at the Runner configuration for their project. You will see we have two categories of Runners: shared and specific.

> Go to Settings, Pipelines.

Shared (Ease, Speed, Efficient Management)

Shared runners are runners that have been provided by the administrators of the GitLab instance, we've been using one as you can see here. By allowing administrators to provide a shared pool, there are a number of benefits:

- Consolidation of infrastructure, whether cloud or on premise. Cluster and credentials can be centrally managed.
- Reduces effort required to set up CI for each team

But there are some cases where an administrator has not provided a shared pool, or they don't meet your needs.

Specific (Self-service, Install)

For these cases, we have the ability for any development team to connect their own Runners. They simply download the Runner, enter in the URL and key, and they are on their way.

Let's take a look at that process now. In the interest of time, I've already downloaded the macOS version.

The first action we want to take is to register our runner:

> gitlab-runner register

During registration we configure a few aspects of the runner:

- We set the URL and token
- Name that we want to appear on the Runner page
- And then any tags we want to specify. Tags allow you to uniquely identify runners with certain properties, for example here we can set osx and xcode8 to identify what we have installed. These are then specified in the .gitlab-ci.yml for the job you want to run.

Runner Architecture (Shell/SSH, VM, Container, Auto-Scaling)

The last choice we have to make is what mode of operation we want for our Runner. The simplest is Shell, where it executes the script on the machine and account it's installed on.

We then have support for working with images and containers, via virtual box, parallels, and of course Docker. The runner will start the specified image, execute the job, and then clean up. These modes are great for shared runners, because the development team can still bring their own base image to start from.

The last mode of operation, is what we call our auto-scaling runner. We support this on Docker Machine and Kubernetes, and here the Runner will elastically process jobs as needed to process the CI queue.

Wrap up (Self-service, Flexible, Scalable)

And that's it for the configuration, we just start our Runner and it is ready to process jobs.

> gitlab-runner start
> Refresh the Runner settings page

This ability of GitLab CI, to allow development teams to set up their own CI infrastructure, is really transformative.

First, self-service: Instead of needing to file a request for a new piece of hardware, wait for the response, justify the changes and cost, have a PO potentially filed... the dev team takes 2 minutes with an old machine from the cabinet an off they go. The dev team is happy and more productive, and the infrastructure is happy they don't have to worry about managing a flock of Mac Minis for the iOS team in their data center.

Second it provides a lot of flexibility. If you need to run jobs on an ARM device, perhaps for Android or Deep Learning, it's as easy as installing the Runner on Android. Need to run something on a mainframe like Linux on Z? Build the runner and away you go. Managing hardware and software dependencies, when something like a container is not possible, could not get any easier.

Last, is scalability. A handful of auto-scaling runners on GitLab.com routinely process over a thousand concurrent CI jobs. If more are needed, simply turn up the dial.

GitLab Pages

The final area we wanted to discuss was GitLab Pages. As you saw earlier with our Pipeline, we pushed our javadocs and code coverage reports here with just a few lines of YML. This site is then hosted by GitLab at a specific domain, making it easy to publish technical content or even entire websites with CI.

Wrap (Flexible: Bash it, script it. Self-service: runners, no plugins. Scalable: SaaS scale CI)

To summarize, GitLab CI is flexible. If you can bash it, you can automate it. Self service runners, no plugins. And SaaS scale CI, with auto-scaling.

SECTION: marketing/brand-and-product-marketing/product-and-solution-

marketing/demo/conference-booth-setup.md

Summary

This page suggests how to set up a conference booth to best use the available media assets to get optimum value out of them.

Main screen(s)

The main screen(s) are the big, usually rented/supplied, usually 48" or more flat screens which are mounted to booth walls. These are meant to grab the attention of folks walking by, and get them to stop and watch long enough to be approached by a booth staff member.

These should NOT show static images (unchanging over time) or they will not achieve their purpose. We have videos that are meant to loop on these screens. These videos can be found in the GitLab Marketing Google Drive under GitLab Booth Videos.

These videos will need to be played off of a laptop so that they can be set to loop.
(even if we find a good way to loop them on the ipads, the ipads are more usefull as click-through demo devices)

Why are there two videos? Well . . .

If there is only one main screen at the booth

1. Download the "GitLab Animation" video (whatever the latest version is).
1. Use QuickTime player to play full screen and in a loop.
1. Make sure the sound is turned off.

If there are two main screens at the booth (hey, it happens)

Screen 1

1. Download the "GitLab Animation" video (whatever the latest version is).
1. Use QuickTime player to play full screen and in a loop.
1. Make sure the sound is turned off.

Screen 2

1. Download the "GitLab Overview for Booths" video (whatever the latest version is).
1. Use QuickTime player to play full screen and in a loop.
1. Make sure the sound is turned off.

Although, this video is silent and shows a combination of slides and product video clips in an end to end use case format.
If you HAVE to play it over streaming, it's also on YouTube

NOTE: Do not display movies, broadcast or cable TV, or non-GitLab streaming or on-demand platform content, as public display of this content requires a public performance license.

iPads

The iPads are meant to be used to start/aid a conversation, as well as to show the click-through demos on. Their advantage is that you can stand next to someone (to the side) and have a personal interaction with them, still with a screen. This is more awkward with a laptop and the need to find a flat surface to put it on.

Here's roughly what we have:

- EMEA - 3 12.9" Pros, logged in using either AppleID gitlabeventsemea@gmail.com or gitlabgitlab@gmail.com
- US Corp - 2 12.9" Pros, logged in using AppleID gitlabconf@gmail.com
- TMM - 4 12.9" Pros, logged in using AppleID gitlabconf@gmail.com, used for development, and brought by TMM for booth duty

Todo: Sync the EMEA ones so they are set up the same way as the others (accounts, etc).

Accounts

- US ipads
  - Pin: 0000 or turned off
  - AppleID login: gitlabnconf@gmail.com (password in Marketing 1Password vault)
- Google login: gitlabconf@gmail.com (password in Marketing 1Password vault)

Setup

Load required software

1. Open App Store (login with above Apple ID)
1. Install Google Drive
1. Install Google Slides

Load required files

There are two possible sets of demos that can be used: regular length, shorts

- regular length - For longer demos (10-15 mins) or lower volume booths.
- shorts - For high volume booths. Meant to run no more than 5 minutes, with a quick overview of how we solve a particular use case.

For either set of files:

1. Open Google Drive
1. login with above Google login
1. go to one (or both) of the above directories
   OR
   go to "Shared with me" menu item or look at the "Shared" view (on the bottom) and find the "Click-through" folder that's been shared and open it
1. To the right of each file are "...". Click on that and select "Make available off-line" for each file you want to have access to

1. For each file, star them (this'll make easier to find later)

Demoing

1. To open, open Google slides
1. Go to the "Starred" view
1. This should show your files, select which one to view
1. When you are not using the ipad, set it to on of the first two slides (shows competitor mess, or shows GitLab stages)
1. In off-line mode, presentation mode doesn't work, but you can show the slides still
1. It can be effective to use iPad pinch/un-pinch to zoom in/out on parts of demo

---
SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/demo/ct-demos.md

Click-through demos can be run in multiple ways:

- Run them off-line (for conferences, practice on the airplane, etc)
- Run them while online, from the files, same as running a presentation
- Or run them online from the web, using the links bellow:

GitLab CI Overview demo (OCT 2020- 12.6)

GitLab Planning to Monitoring (Aug 2019 - 12.0)

<!--T Downloadable macOS
Downloadable Windows-->

Create Kubernetes cluster (Aug 2019 - 12.0)

<!--Downloadable macOS
Downloadable Windows-->

Merge Request Approval API (July 2019 - 12.0)

<!--Downloadable macOS
Downloadable Windows-->

Agile Project Management (June 2019 - 12.0)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vR5EWjzgr-Qe-cYdMj4WJnnVDxKyoyirMv0OfOJWbgCzevJoLGTXnbKDYm2TUhpdPAkh-nTcucIgZKx/embed?start=false&loop=false&delayms=3000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Cross-project Pipeline Triggering and Visualization (May 2019 - 11.10)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vRZaaHmlJcBQS56SSS1QKyvLJQayJ31ZXqlAJMzQOMckbHt0dSj0KBN2bzg6lwny-lqfvhfhOl7tK8H/embed?start=false&loop=false&delayms=3000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Secure Capabilities (March 2019 - 11.8)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vTmtpY92Uun3YsixCJHZu7yt69M9rFB5SuFRSglOBRXoFNTKBdgPZpE4JBTl3LtAAX1zS4zQdBdD6Ga/else&loop=false&delayms=60000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file
Sales enablement recording (of shorter version)

Secure Capabilities (short, guided) (March 2019 - 11.8)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vQF2Neh1h0vFwMapLvhpprbZZVaxbtnVvTP69xd6YNGreW5dZ43w4w5qQTmYNewmI-3pViilsvbIcX/embed?start=false&loop=false&delayms=60000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>
G-slides file
Booth Demo Auto Play link
Sales enablement recording

Auto DevOps (Oct 2018 - 11.3)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vTNeBj0TJWca1PVwccxUTXWDWYqaIyB6Q1fRYCfjtMzeK8DtpmAcG1o6ipFBi-lhYKVTAA9kYBWyKKu/embed?start=false&loop=false&delayms=600000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>
G-slides file
Sales enablement recording (of shorter version)

Auto DevOps (short, guided) (July 2018 - 11.0)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vRQ4xltHhYROzgbo7exF6BwR09jvPmyFzR4XvjdlpYMRqT4dctx61XCkLjfR-8sq6QyOsoEFBBJjJh/embed?start=false&loop=false&delayms=600000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>
G-slides file
Booth Demo Auto Play link
Sales enablement recording

Auto DevOps - Setup (GKE) (Oct 2018 - 11.3)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vQIPrinsvTG1s6ppnUWqSY-fpHnxe6oAwM
7g91uBl8Mx3EYQhaejlKUF9c3GtagIhzwg-8dJlIrNgw/embed?start=false&loop=false&delayms=600000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>
G-slides file

Auto DevOps - Setup (EKS) (Oct 2018 - 11.3)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vRjhO2VvKfgWbObwi5APm18gxQrzQk5vvAARD-4vLQeT0NbkrSuP3t4sTVylRYZOD6kINLr37HmtHA/embed?start=false&loop=false&delayms=60000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>
G-slides file

Live (instructions)

Planning to Monitoring (formerly i2p)

Planning to Monitoring (formerly i2p)

Highlights GitLab's single platform for the complete DevOps lifecycle, from idea to production, through issues, planning, merge request, CI/CD, and monitoring.

CI/CD Deep Dive

CI/CD Deep Dive

Provides a more in-depth look at GitLab CI/CD pipelines.

Integration Demos

Integration Demos

Demonstrations which highlight integrations between GitLab and common tools such as Jira issues and Jenkins pipelines.

Conference Booth Setup

We've started using iPad's to run click-through demos at conferences, and also we now have a main screen video to loop that shows our key slides and product clips. You can find details about both of these on the conference booth setup page.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/demo/executive-demo.md

Scaled Agile Framework (SAFe)

The scaled agile framework has evolved to be a common approach for large enterprises adopt agile delivery practices at scale where they need to manage governance, coordination, and cross project collaboration.

Before explaining how GitLab can support SAFe, a brief overview of the GitLab project and portfolio model help. This slide deck is where we have been collaborating internally about how GitLab can support the Scaled Agile Framework.

GitLab Project Management

GitLab "Project"

The project in GitLab is the core building block, where work is organized, managed, tracked and delivered. A project in GitLab enables the team to collaborate and plan their work in the form of Issues (use cases/requirements), Issue boards (Kanban), and Milestones (Sprints).

The GitLab Project is much, much, much more than simply project management. The GitLab project unlocks the power of an industry leading source code management repository and a CI/CD pipeline. The Merge Request is the linkage between the issue and the actual code changes. The merge request captures the design, implementation details (code changes), discussions (code reviews), approvals, testing (CI Pipeline), and security scans.

Issue

The issue in GitLab is the fundamental planning object. It is where the team documents the use case in the description, discusses the approach, estimates the size/effort (issue weight), tracks actual time/effort, assigns work, and tracks progress. Labels enable the team to tag issues, track status, and associate issues with different initiatives.

Boards

Boards provide a flexible and dynamic approach to visually manage a project. Here, teams can manage their backlog of work, prioritize the items, and then move the issues to the team or specific stage in the project. Each list in the board calculates the total size (weights) of the associated issues, enabling the team to understand how much work is assigned at any given

time.

Milestones

Milestones establish a target date for a sprint or specific bundle of issues and code changes to be delivered. The milestone enables the team to either set a specific Start and Stop for the work, as in a Sprint, or the milestone could be a fixed point in time.

Labels

The label in GitLab is a flexible and powerful mechanism to tag and track work. Labels are used to define columns in the issue boards and make it easy to search and find issues or merge requests related to a common theme.

GitLab Groups

In order to manage multiple projects (portfolios of projects), the GitLab Group is the entity that enables strategic planning and tracking of business initiatives through delivery. At the Group Level, you can manage sub-groups, projects, epics, milestones, roadmaps and group level boards.

Epics

In order to track groups of related projects and issues, the GitLab epic gives product owners and leaders the ability to link and manage work over an extended time frame. An epic can span multiple milestones and makes it easier to manage the overall flow and priority of work.

Milestones

While milestones at the project level often align to sprints, at the group level, milestones can be created for all the projects and sub-groups within the group. This way, teams can stay in synch with each other and focus on common release targets.

Roadmaps

The roadmap is a visual representation of the various epics for the group. The roadmap view can be filtered by label and organized by start / stop date of the epics in order to visualize the sequence of work. At this point, GitLab doesn't create dependencies between issues or epics.

Group Boards

The group level issue board makes it possible for oversight and governance of the projects and sub groups. This view, makes it easy to see how specific issues are flowing through the

lifecycle and to understand the overall capacity of the teams.

Scaled Agile Framework

The scaled agile framework is used by many large enterprises to define, organize, and synchronize the work of multiple agile teams. Designed to help enable coordination, collaboration, governance and oversight of multiple agile teams in complex environments.

GitLab's structure of Groups, SubGroups and Projects makes it possible to model and support different variations of the Scaled Agile Framework.

To support the entire model (the 4 layer SAFe model), an organization would start with a Group to represent the Portfolio planning work. Then the Large Solution part of the model would be a Sub Group. The Programs would each be listed as SubGroups below the Large Solution, and finally the agile teams would work in GitLab Projects, each modeled as part of "Programs".

Team

The agile team is the foundation of the Scaled Agile Framework, it is where teams organize, plan, and actually deliver new features. In GitLab, the project is the fundamental team work area. In GitLab projects teams manage their backlog of use cases (issues), assign work (boards), collaborate on features (issue discussions), develop code (merge requests), review changes (merge request discussions), and integrate and deploy applications (CI/CD pipelines).

Team backlog

GitLab issues and issue boards enable the project team to capture, manage and prioritize their backlog. They can prioritize their issues directly in the issue board, or they can use labels to establish overall priority of issues.

Scrum

Project level milestones are used to define time based sprints. Issues are added to the milestone and then GitLab enables burndown charts to visualize the progress of the team in closing issues and delivering new features.

Team Kanban

Project issue board helps teams to visualize and manage the flow of their work, with the ability to track the overall 'weight' of all the issues assigned to any specific stage in the board.

Large Solution

In order to coordinate how multiple dimensions of a complex solution are delivered, the Large Solution layer of the scaled agile framework is designed to facilitate cross project coordination. In GitLab, a subgroup can enables oversight and coordination of both projects and additional subgroups.

Solution Intent

Solution Intent

Solution Train

Solution Trains enable cross project collaboration and coordination of work to meet specific release targets. In GitLab, the Group boards and labels are used to define specific stages in the solution train and to track how project level issues and merge requests are progressing toward being ready to release.

Portfolio

At the top of the Scaled Agile Framework is where portfolio and strategic planning drive decisions about future investments and business objectives. In GitLab, the Group is able to contain both Projects and Subgroups, to enable reporting, tracking, and management of strategic initiatives.

Strategic Themes

Strategic themes are effectively long term goals and objectives at the portfolio level. In GitLab, the Epic at the group level can be used to define the overall theme.

Epic

The epic is a way to package and organize work required to deliver specific business value. In GitLab, Epics are a way to strategically organize work from multiple projects.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/demo/gke-setup.md

Video

The video below shows installing GitLab on Google Kubernetes Engine (GKE). For the DevOps lifecycle, please refer to the
sales demo.

<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/HLNNFS8baw" frameborder="0"

allowfullscreen="true"> </iframe>
</iframe>
</figure>

Preparation

- You need a Google Cloud Platform account, GitLab employees will have this. Ensure you are logged in with your GitLab account.
- Login to Google Kubernetes Engine.
- GitLab employees should use the gitlab-demos project. Others should select or create a project to work in.

  - URL: https://console.cloud.google.com/kubernetes/list?project=gitlab-demos

- If you've run through the demo before but didn't clean up your demo cluster(s), do so now.
- This script assumes the make-sid-dance.com domain, but you should either:

  - Pick the least-recently used domain from the Google Doc (internal only). (Let's Encrypt limits SSL cert creation on a weekly basis, so rotating usage helps reduce hitting the limits), or
  - Buy a new domain for your demo and substitute throughout the script.

    - Create DNS Zone to let Google manage DNS for you.
    - Click Registrar Setup to see what name servers to use.

- Disable desktop notifications (on a Mac, top-right corner, option click).
- Open up new browser window so the audience doesn't see all your other open tabs.
- Resize your browser window to something reasonable for sharing. 1280x720 is a good option. <https://handbook.gitlab.com/handbook/marketing/brand-and-product-marketing/product-and-solution-marketing/demo/720p.scpt> is a handy AppleScript if you're on a Mac and using Chrome. Add it to your User Scripts folder and show the Script menu in your menu bar, and it'll be really easy to trigger.
- Consider just sharing web browser window so the audience isn't distracted by notes or other windows.
- If displaying full-screen, go to 'Displays' settings, Resolution: Scaled, Larger text.
- Consider opening this page on an iPad that has screen lock disabled.

CLI setup

- On macOS, install brew for all the things
- ruby -e "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install)"
- You need to have the Google Cloud SDK installed. e.g.
- Install Brew Caskroom
- brew install caskroom/cask/brew-cask
- Install Google Cloud SDK
- brew cask install google-cloud-sdk
- Run gcloud components install kubectl
- Install helm

  - brew install kubernetes-helm

- Before each demo, run sudo gcloud components update; gcloud auth application-default login, saving you time from doing this in the middle of the demo.

Set up a container scheduler cluster

We're going to install everything from scratch and we'll start by creating a new container cluster. Today I'm going to use Google Kubernetes Engine.

- Create cluster (or open GKE, pick gitlab-demos project and click Create cluster).

We'll name this cluster make-sid-dance and have it created in the us-central zone. I will bump up the machine type to have 2 virtual CPUs for performance reasons, but I'll drop it down to 1 node. A real cluster should have 3 or more nodes for better availability.

- Name the cluster after your domain name (e.g. make-sid-dance).
- Note the Zone field should read us-central1-, and will have a letter on the end. This letter does not matter.
- Change the number of vCPU in Machine type to 2 vCPU.
- Change the Size to 1. Note: The demo will run fine on more nodes, if desired
- Click the Create button at the bottom of the page.
- End Advanced options in the Security section select Enable legacy authorization

Now we need to get an external IP address for the demo so that we can use a domain name and Let's Encrypt for SSL.

- Navigate to VPC Network.
- Select External IP addresses from the menu on the left.
- Click Reserve static address at the top of the page.
- Set the name to match the name used for the cluster (e.g. make-sid-dance).
- Set the Region to us-central1 to match the Zone where you made the cluster.
- Warning: The external IP must not be attached to anything yet. This will happen automatically in a later step.
- Click the Reserve button at the bottom of the page.

We'll now create a wildcard DNS entry for our demonstration domain, pointing to the IP we just created.

- Copy the External Address from the list, from the line containing the name you used.
- Navigate to Network services
- Click Cloud DNS from the menu on the left.
- Click on the Zone that has the name of the domain to be used for the demo. (e.g. make-sid-dance-com)
- Click on the Add Record Set button at the top of the page.
- Set the DNS Name to .
- Set the IPv4 Address to the clipboard contents (the External Address you just copied).
- Click the Create button at the bottom of the page.

Now that we have created the cluster and configured a domain, we can go back and check on our cluster.

- Navigate to Kubernetes Engine.
- Confirm a green checkmark.

Good, our cluster is ready for us to use. Let's connect to it.

- Click on the Connect button for your cluster.
- Click the copy icon to the right of the gcloud container ... entry. It looks like two overlapping white boxes.
  - Run this command:

```
shell
gcloud container clusters get-credentials makesiddance-com \
--zone us-central1-a --project gitlab-demos
```

- Switch to the Terminal window, paste this command in and run it.

Set up GitLab itself

Now that we have our cluster configured, we're ready to install GitLab. To do this, we'll need the base domain name, the external IP address we just configured, and an email address to use with Let's Encrypt. Then we use helm to install all the necessary components.

- helm init
- helm repo add gitlab https://charts.gitlab.io
- helm upgrade -i makesiddance --namespace gitlab --set baseDomain=makesiddance.com,baseIP=192.168.1.1,legoEmail=you@gitlab.com,provider=gke gitlab/gitlab-omnibus (Replacing baseDomain, baseIP with External Address from above, and legoEmail as appropriate.)

Alternate instructions for GitLab EE

- Go to /free-trial/ and request a trial license for GitLab EE
- Wait for email
- Download license to ~/.gitlab-license
- Install helm chart, adding the gitlab and gitlabEELicense options:

```
console
export LICENSE= cat ~/GitLab.gitlab-license
helm upgrade -i makesiddance --namespace gitlab --set baseDomain=makesiddance.com,externalIP=19
2.168.1.1,legoEmail=you@gitlab.com,provider=gke,gitlab=ee,gitlabEELicense=$LICENSE
gitlab/gitlab-omnibus
```

Now let's check if our gitlab service is up, and wait for it if not.

- kubectl get deployment -w gitlab --namespace gitlab
- Wait until Available shows 1.

Optional filler

- kubectl proxy
- Go the Kubernetes Dashboard at http://localhost:8001/ui
- Change the Namespace drop-down on the left. Change it from default to All Namespaces
- Click on Workloads on the left.

GitLab is now deploying, and we can watch the status from the Workloads page in the Kubernetes dashboard. We'll watch here for all items to have a green checkmark showing that they have completed. This process can take a few minutes as GKE allocates resources and starts up the various containers. You can see here there are several containers. The main GitLab container has the Rails app, but also Mattermost for Chat, the integrated Docker Registry, and Prometheus for monitoring. Then there are separate containers for Postgres and Redis and the autoscaling GitLab Runner for CI and CD. This is everything you need for the DevOps lifecycle on Kubernetes.

While we're waiting: In the next demo, I'll take you through everything you need to take ideas to production, including chat with Mattermost, issues and issue tracking, planning with issue boards, coding with terminal access, committing with git version control, merge requests for code review, testing with continuous integration, getting peer reviews with live review apps, continuous delivery to staging, deploying to production directly from chat, cycle analytics to measure how fast you're going from planning to monitoring, and lastly, Prometheus monitoring of your GitLab instance. With GitLab, everything is integrated out of the box.

What takes 10 minutes in this demo would take days if you're not using GitLab and have to integrate different tools. Not only is GitLab faster to set up, but it is also more convenient to have everything in one interface. Developers want to work on creating a great product, not on learning and maintaining the integrations between theirs tools.

If there is more time talk about what a review app is and what cycle analytics are.

- Wait for gitlab pod to go to green or deployment to show available

Looks like our deployment is finished. Let's check out GitLab...

- Open a new tab with gitlab.make-sid-dance.com (Adjusting the URL to the domain you used for this demo)

Boom, we've got a shiny new GitLab installation!

Set root password

Before we get too carried away, we need to secure the root account with a new password.

- Set password for root user (You don't need to actually log in as root, but you can)

Cleanup

- Before you delete the cluster, delete all of the underlying services/pods/etc. using the CLI.
- If you accidentally delete the cluster using the web UI, make sure you:

  - Look for persistent disks that need to be deleted manually.
  - Look up the external IP you used, find the ID of the load balancer it is forwarding to, then find that ID in the list of load balancers. Delete the load balancer.

- Release the static IP.
- Delete all orphaned disks

Troubleshooting

Various failures block Let's Encrypt, and thus GitLab

There are several scenarios which can cause deployment failures due to issues surrounding the kube-lego-nginx and the Let's Encrypt (LE) ACME process. The easiest way to find these errors is checking the logs of the kube-lego-nginx service in the kube-lego namespace of the dashboard for your Kubernetes cluster.

1. Let's Encrypt top-level domain request rate limit exceeded

The failure mode here is the most vague from the logs, however it occurs when you have exceeded the number of certificate or renewal requests allowed for a single TLD. Please see their documentation regarding this.

1. Unresolvable DNS

If your DNS records are not correctly configured, the Let's Encrypt servers may not be able to resolve your domain when the ACME requests are filed against it. Let's Encrypt performs a reachability test that depends on valid, resolvable Fully-Qualified Domain Names. You must confirm that your entry DNS is functional, and has propagated. You can do this by using an external host (anywhere not directly querying your primary DNS where this record is present) to ping test.my.tld where my.tld is the domain name you are using. Because you should have configured a wildcard record (.my.tld), test.my.tld should resolve to that address.

1. Host not responding (reachability)

This has been observed as a failure of the LoadBalancer to be properly connected to the reserved statis external IP address. There are a few methods of failure here, but the primary cases are:

- Unable to assign due to prior assignment.

Either an existing use, or a failure to fully remove the prior deployment. This has been seen in both scenarios by GitLab personnel. If you are re-creating a previous deployment, you need to wait a short period and/or confirm that the previously used GCP Networking LoadBalancer has been removed. You can manually remove these if you do not wish to wait for GCP to catch up with the de-provisioning.

- Unable to assign due to incorrect region.

If you inadvertently create a GKE Kubernetes cluster in a region that is not the same as the static IP address you are attempting to use, your deployment will fail to attach to that IP address, and result in the inability to listen and respond to requests.

General notes

Creating connection to your cluster from kubectl

- Navigate to Container Engine.
- Click on the Connect button for your cluster.
- Click the copy icon to the right of the gcloud container ... entry. It looks like two overlapping white boxes.
  - Run this command:

```
shell
gcloud container clusters get-credentials makesiddance-com \
--zone us-central1-a --project gitlab-demos
```

- Switch to the Terminal window, paste this command in, run it.
- run kubectl proxy

Logs

- You can find logs for each pod in the Kubernetes Dashboard

  - Select Namespace you want to see logs for
  - Navigate to Pods
  - Select Pod you want to see logs for
  - Click on View logs

- You can check logs from CLI using kubectl as well

```
console
kubectl get namespaces
kubectl get pods --namespace=<NAMESPACE>
kubectl logs <POD--namespace=<NAMESPACE>
```

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/demo/i2p.md

<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/nMAgP4WIcno" frameborder="0" allowfullscreen="true"> </iframe>
</figure>

Note: This is the latest video. Work to catch up these demo instructions to match the video is

underway.

Overview

Today, modern software development teams need version control for everything, automated testing, support for complex build and deployment configurations, and end-to-end visibility and traceability so they can work to improve their software development and operations over time. But for most teams, getting this tooling right is incredibly difficult.

This demonstration will highlight GitLab's single platform for the complete DevOps lifecycle, from plan to
monitor, through issues, planning, merge request, CI, CD, and monitoring.

If you encounter issues replicating this demo on GKE or on your own Kubernetes cluster please open an issue. We're still
working to improve this demo further, please see all open idea-to-production issues.

Preparation

> - Disable desktop notifications (on a Mac, top-right corner, option click).
> - Open up new browser window so the audience doesn't see all your other open tabs.
> - Resize your browser window to something reasonable for sharing. 1280x720 is a good option. <https://handbook.gitlab.com/handbook/marketing/brand-and-product-marketing/product-and-solution-marketing/demo/720p.scpt> is a handy AppleScript if you're on a Mac and using Chrome. Add it to your User Scripts folder and show the Script menu in your menu bar, and it'll be really easy to trigger.
> - Consider just sharing web browser window so the audience isn't distracted by notes or other windows.
> - If displaying full-screen, go to 'Displays' settings, Resolution: Scaled, Larger text.
> - Consider opening this page on an iPad that has screen lock disabled.

GitLab installation

There are four options:

1. Log in at gitlab.i2p.online (for GitLab sales people)
1. Set up a cluster on Google Kubernetes Engine (GKE)
1. set up a cluster on Azure Container Service (ACS)
1. Offline local demo environment with Minikube

If you need to use a license, GitLab team-members can request a license.

<!-- Enable Auto DevOps

> Log in as administrator
> Go to Admin Area (wrench icon)
> Click on Settings

> Under Continuous Integration and Deployment, click on Enabled Auto DevOps (Beta) for projects by default
> Click Save
> Log out as administrator-->

Project set up

Cleanup

> - Delete previous GitLab groups you made last time
> - Delete previous Kubernetes namespace for projects (kubectl delete namespace minimal-ruby-app-<project-id>)
> - Delete previous Mattermost team or at least leave Mattermost team

Create a user

Note: If using a shared demo, each demo-giver only needs to do this once.

Now let's register a new user in our GitLab server.

> - Click Register
> - Create a user with your name and email address (no verification sent)

Create a group (optional, if showing Chat)
<!-- Maybe re-use to show how groups work instead of for Mattermost -->

We've got GitLab running and we're logged in. Since we'll want to work as a team, we'll create a GitLab group. Groups allow you to organize projects into directories and quickly gives users access to several projects at once. You can even nest subgroups under groups to match your org structure. Let's give ours a unique name.

> - Click hamburger menu in top-left corner > Groups
> - Click New Group
> - Give the group a unique name, perhaps the name of the company you're demoing to, or a made up name (all lowercase, no spaces or special characters other than -)
> - Change Visibility level to Public
> - Click Create group

Create a project

Note: If you've given the demo before, make sure to delete the minimal-ruby-app project first.

Now let's create a new project. I'll import from a really simple example app just to save myself some typing.

> - Click New Project
> - Under Import project, click Repo by URL
> - Set Git repository URL to https://gitlab.com/auto-devops-examples/minimal-ruby-app.git
> - Set Project name to minimal-ruby-app

> - Make it public

Set up GitLab Auto DevOps

Now we're ready to configure GitLab Auto DevOps, which is the easiest way to configure GitLab CI/CD with a bunch of best practices already built in. Let's go to the CI/CD settings. We just need to enable Auto DevOps and give it a base domain.

> - Go to Settings > CI/CD
> - Expand Auto DevOps
> - Check the box Default to Auto DevOps pipeline
> - Change domain to i2p.online (or the base domain you are using) by creating the KUBEINGRESSBASEDOMAIN variable
> - Leave the default of Continuous deployment to production
> - Click Save changes

Now we see it kicked off the first pipeline, which I'll dig into later, but for now, it's great to know that without any effort, we've got a fully functional CI/CD pipeline.

Great, that completes our setup.

Project permissions (optional)

Okay, so everything we need to bring an application from plan to monitor is set up. But let's assume you want to safeguard your source code before handing this over to your developers. I'll take you through a few key ways you can outline project permissions and manage your team's workflows.

User roles and permissions: Since this is a public project, we'll want to ensure that we have a way to manage what actions each team member can take. For example, we may want only certain people to be able to merge to master or to be able to adjust the CI project configuration.

Change a user's permission level: In GitLab, permissions are managed at a user and group level and they apply to projects and GitLab CI. We have five different role types, so you can set granular permissions and keep your code and configurations management secure. To save your admins time and the headache of managing multiple logins, GitLab integrates with your Active Directory and LDAP. You can just connect GitLab to your LDAP server and it will automatically sync users, groups, and admins.

Project settings: In addition to permissions, we also have features to help you manage the team's workflow and bake quality control into your development process.

Navigate to project settings for protected branches: It's no secret that code review is essential to keeping code quality high. But when the team is on a deadline, there could be an incentive to skip code review and force-push changes. Therefore, in addition to permissions, we also allow you to identify protected branches to prevent people from pushing code without review. The master branch is protected by default but you can easily protect other branches, like your QA branch, and restrict push and merge access to certain users or groups.

Navigate to project settings for merge request approvals: If you want to take code review a step further and ensure your code is always reviewed and signed off by specific team members, you can enable merge request approvals. GitLab lets you set the number of necessary approvals and predefine a list of approvers or approval groups that will need to approve every merge request in the project.

Permissions, merge request approvals, and protected branches help you build quality control into your development process so you can confidently hand GitLab over to your developers to get started on turning their ideas into a reality.

Plan to Monitor (formerly Idea to Production) (main demo)

Issue (Plan)

Let's create our first issue, starting from an issue board.

> - Go to Issues > Boards
> - Click on + in Backlog column
> - Set Title to Make homepage prettier
> - Submit issue
> - Click X to close issue if needed

Board (Plan)

Inspiration is perishable, so let's pick this one up right away. Since this is our first time, we have to add a couple columns here to match our workflow. These columns are fully-customizable and you can have multiple Issue Boards per project to help your team organize their releases. I'll just add the default "To Do" and "Doing" columns.

> - Add default lists

There. Now we can just move the new issue from the backlog into the Doing column, because we want to resolve this issue right now.

> - Drag issue from Backlog to Doing

Commit (Create)

Now let's get coding! I'll click through to the issue and create a new merge request. This creates a new branch for me, starting with the issue number to keep them associated and automatically specifies that this merge request will close the issue once it's merged.

> - Click on issue title in issue board card
> - Click Create merge request

Clicking through to the branch, I see that it's a really simple app. Viewing the files I can see it's basically just Hello World, but with some random timings to make monitoring more interesting. I'm going to go ahead and update the message

and remove the TODO comment using the WebIDE.

> - Click on branch name (e.g. 1-make-homepage-prettier)
> - Click on server.rb
> - Click WebIDE button

The WebIDE, which is a part of GitLab gives me a development environment without any configuration necessary on my client. I can edit multiple files, stage my changes, preview them, commit, and view the resulting pipeline statuses all from the IDE.

> - Change Hello, world! to <h1>Hello, world!</h1>
> - Delete the TODO: use HTML line
> - Click the Commit... button

When I commit changes the WebIDE immediately shows me the diffs so I can do a quick check of my changes. I can then stage the changes I want to commit, add a commit message, and then I have some options about branching. Since we already have a merge request started on this branch I'll just commit to the existing branch.

> - Double-click on server.rb to move it to Staged changes
> - For the commit message type Added HTML
> - Make sure Commit to <your branch name> branch is selected (eg. Commit to 1-make-homepage-prettier branch)
> - Click the Commit button
> - If popup asks to show notifications, click Allow

Build Stage (Verify)

After the commit, we can see in the WebIDE that it automatically kicked off the CI/CD Pipeline that will test our contributed code. We can see the pipeline contains many stages including Build, Test, Review, dynamic testing, and then Cleanup.

> - Click on rocket icon in the top right of the WebIDE

In the build job, we build the Docker container image and push it to the built-in container registry.

> - Click on the View log button in the Build stage
> - Slide open the resulting pane to show more at once

If we didn't have a Dockerfile, it would have used Heroku buildpacks to detect the language and framework and build an appropriate Docker image.

Runner progress

(optional: if CI/CD is taking a while)

While it's running, we can head back to our Kubernetes console to see that our GitLab Runner is working directly with Kubernetes to spawn new containers for

each job, as they are needed. It even creates a namespace for the project, providing isolation.

> - Go to Kubernetes
> - Change Namespace to default
> - Click on Pods
> - Change the Namespace drop-down to minimal-ruby-app-<number>
> - Click on Pods

Test stage (Verify)

Let's look at this same pipeline from a different view.

> - Click the < View jobs button to get back to pipeline view
> - Click the Pipeline (should be a link) at the top of the panel

In the Test stage we see six jobs...

Code Quality (Verify)

> - Click codequality

The codequality job, runs static analysis on your code to look for stylistic and other quality problems. Catching these types of problems early makes them 100's of times cheaper to fix, and helps keep technical debt away.

Container scanning (Security)

(optional: requires GitLab Ultimate)

> - Click back (to pipeline view)
> - Click containerscanning

The containerscanning job analyzes your application environments (your containers) for known security vulnerabilities. This makes sure that your application is running in as secure an environment as possible from the start.

Dependency scanning (Security)

(optional: requires GitLab Ultimate)

> - Click back (to pipeline view)
> - Click dependencyscanning

The dependencyscanning job finds and reports on security vulnerabilities in the libraries your application is dependent on. This catches reliance on vulnerable libraries and provides the opportunity to correct them before development gets too far.

License compliance (Verify)

(optional: requires GitLab Ultimate)

> - Click back (to pipeline view)
> - Click licensemanagement

The licensemanagement job analyzes your application dependencies and highlights the presence of licenses you don't want to use, or new ones that need a decision. This catches reliance on libraries with unwanted licenses early, before it is more costly to change.

Static Application Security Testing (Security)

(optional: requires GitLab Ultimate)

> - Click back (to pipeline view)
> - Click sast

The sast job runs static application security testing on your code to look for known security vulnerabilities. Catching security vulnerabilities early means less work to resolve the issues, less costs, and safer code from the start.

Test (Verify)

> - Click back (to pipeline view)
> - Click test

The test job detects the language used, again using Heroku buildpacks, and runs the language appropriate tests you've defined. In this case it's Ruby, so it runs rake test.

Review (Create)

Review apps

When the tests pass, it automatically creates a temporary review app in our Kubernetes cluster.

> - Go Back (to pipeline view)
> - Click on Review

Here we see a bunch of steps it's doing automatically for us with the highlight being a deployment to Kubernetes, using our default Helm chart. If the app had a Helm chart in the project itself, it would have used that custom chart instead. Or I can even add a project variable to point to another chart. But here, we're using the default.

To see the beauty of the review app, let's go back to the merge request.

> - Click on the commit ID above the pipeline to get to the commit page
> - From the commit page, go to the linked MR (begins with a "!")

Here we see a new line showing us that it was deployed to the review app, and here's the URL to actually see my changes running live. This is great because I don't want to trust reading the code; I want to see it live in a production-like environment and this review app provides that.

> - Click on external link to review app

So this is what we just changed, and any new changes pushed to our branch will automatically update the app.

Now back to the merge request...

> - Close review app tab
> - Click Expand on Code quality

We see there's a new line for the code quality. We see that it likes that we removed the TODO. If we made things worse, it would show up here as well.

And there's a new line for the security scanning, showing that no security vulnerabilities were detected.

<!--
Debugging (Terminal)

Now if there were any problems, for example differences between development and production, and you don't want to keep testing changes by pushing them to source control, we could debug those problems right here. By clicking the web terminal button we get a command prompt in the same container as our application.

> Click on review/1-make-homepage-prettier
> Click Terminal button (on the upper right, 1st on right)

All our files are here. Let's edit the server.rb file.

> ls
> vi server.rb
> i (to insert)
> Update text to Corrected Hello World!
> esc (to go back to normal mode)
> ZZ (to save and close)

Now we've saved the changes, let's restart the server.

> killall ruby

And now we can view the web page live to see how we like the changes.

> Click external URL link on top right (2nd from right)
-->

Code review

At this point we'd usually ask for another developer on the team to review our merge request. They can see the exact code that has changed, comment on it, and we'd see a thread of the discussion, as well as get an email notification, of course.

> - Close review app tab
> - Close environment tab
> - Click on Changes
> - Click on a changed line to show ability to comment
> - Comment "Looks good", Comment
> - Go to Discussion tab to see comment

Merge to master

This all looks great so let's remove the WIP status and merge the changes into the master branch.

> - Click Resolve WIP status
> - Check Remove source branch
> - Click Merge

Production (Package & Release)

> - Click on CI/CD > Pipelines
> - Click on top pipeline

Now we see it's kicked off a new pipeline. And inside this pipeline, it looks familiar, but a little different. Instead of creating a review app, this one is deploying right into production. This is continuous deployment at its best.

Environments with deployment history

> - Go to Environments

Going to the Environments page, I see production listed here and the last deploy happened less than a minute ago. And I can easily click through to see what it looks like.

> - Click external URL link (left-most button)

There we go! We've got our new formatting changes; all the way from plan to monitor!

> - Close production app tab

Scaling and Deploy Boards (Configure) (optional: requires GitLab EE)

Now, there's more to this page. Here I see the deploy board for production.

Right now it's only showing a single pod, but let's go and scale that up. I'll go to CI/CD settings and add a project variable to set the PRODUCTIONREPLICAS to 4.

- Settings -> CI/CD
- Expand Secret variables
- Set key to PRODUCTIONREPLICAS
- Set value to 4
- Add new variable

Now let's go back. And I can redeploy.

- Click back twice to get back to environment list or go to CI/CD > Environments
- Click Re-deploy

Soon we'll see the deploy board update in realtime as the fleet rolls out, and we can wait a bit to see the deploy finish.

Monitoring (Monitor)

So that's a high level status of the deploy, but how about monitoring the ongoing health of your app environments? Clicking on the graph icon, I see response metrics, with latency, error rate, and throughput, and system metrics, with CPU and memory usage, and all of these are taken from the built-in Prometheus monitoring, the leading open-source monitoring solution. There's not much to see right now, but this will show the last 8 hours so you can monitor how your app is doing. There's lines for each deploy as well, so you can corelate changes in performance caused by recent deployments. Application performance monitoring can help your team be more strategic, preventing errors vs. simply reacting to them. Imagine if your application monitoring tool could help you avoid pushing poor-performing code in the first place, saving your business future downstream costs? That's exactly where we are heading.

Closing the loop (optional)

Let's close the loop here and go back to our merge request. Since it's already been merged, we have to look for it in the right tab.

> - Go to Merge Requests > Merged
> - Click on top merge request

On the merge request, we now see another status showing that this code has indeed been deployed to production. This is great for managers looking at closed merge requests to know if they've actually been deployed or not. But we go further and show feedback about your application's performance, right on the merge request, telling you how much your memory usage changed before and after the merge request was deployed.

Custom pipeline - staging and canary (optional: requires GitLab EE)

So that covers Auto DevOps from build, test, code quality, review app, deploy to production, and monitoring. Pretty awesome. But what if you want to do something differently?

Well, here I'll go to manually set up CI, but instead of having to start from scratch and re-create all that we just saw, I can pick Auto DevOps from the templates, and import the whole thing.

> - Go to Overview
> - Click on Set up CI
> - Pick Auto DevOps template

There's a lot in here, but it's not as scary as it first looks. And there's some helpful tips in here already. Say I'm not ready for continuous deployment to production, I can just uncomment out this staging job. And if I want to add canary deployments, I uncomment this one. I then just need to uncomment this one last line to make my deployments to production manual and I'm ready to go.

> - Remove leading . from staging job
> - Remove leading . from canary job
> - Remove in front of when: manual line in production job

So I save this into master. And now I see another pipeline is kicked off. This one again looks familiar, but there are two new stages for staging and canary.

> - Commit changes to master
> - Go to CI/CD > Pipelines
> - Click on top pipeline

Let's just wait for staging to finish deploying. There, now we can go back to environments and see there's a new staging environment. With this configuration staging is going to be automatically updated with the latest changes.

> - Go to CI/CD > Environments
> - Click on staging external URL
> - Close staging tab

But production is no longer going to update automatically. When we're ready, we have to manually promote from staging to production. But we're not going to ship directly to the entire production fleet. GitLab supports canary deploys, basically letting you deploy to a smaller portion of your fleet to reduce risk. So here I'll click on the Canary manual action to start the deployment. And I can even see my canary deployment happening in the deploy board!

> - Click on staging manual action dropdown, select Canary
> - Expand staging deploy board

Now there's one new pod in production running the new canary code, and it's taking a portion of production traffic, but the rest of the pods are still running the old code. This is a great way to reduce risk in deployments.

When we've validated that the canary is working as expected, we can then go and ship it completely to production. Let's go ahead and do that now.

> - Click on production manual action, select Production
> - Wait for deployment to finish
> - Click on production external URL

Great, now it's in production!

So whether you want continuous deployment to production, or a continuous delivery flow that's more under your control, or even if you want something else altogether, we've got you covered.

> - Close production tab

Feedback (Cycle Analytics) (optional)

To help you spot bottlenecks in your development process, GitLab has a built-in dashboard that tracks how long it takes the team to move from plan to monitor.

> - Go to Overview > Cycle Analytics

Here we can see some metrics on the overall health of our project, and then a breakdown of average times spent in each stage on the way from plan to monitor. So far, we're doing amazingly well, by completing a release cycle in minutes.

This is great for managers looking to better understand their company's release cycle time, which is critical to staying competitive and responding to customers and changing market needs.

Instance Monitoring (optional)

Not only does Prometheus monitor your apps, but it monitors the GitLab instance itself. Let's go to the Prometheus dashboard.

> - Visit Prometheus https://prometheus.i2p.online (Change the domain to match the domain used for your GitLab installation)

Let's look at a couple simple queries that show how your GitLab instance is performing. Here's our CPU usage:

> - Copy 1 - rate(nodecpu{mode="idle"}[5m]) into the Expression bar; hit enter.
> - Click Graph

And then memory usage:

> - Copy (1 - ((nodememoryMemFree + nodememoryCached) / nodememoryMemTotal)) 100 into the Expression Bar; hit enter.

Conclusion

So that's it. In less than 10 minutes, we took an idea through the complete DevOps lifecyle, with issue tracking, planning with an issue board, committing to the repo, testing with continuous integration, reviewing with a merge request and a review app, debugging in the terminal, deploying to production, scaling the application, application performance monitoring, and closing the feedback loop with cycle analytics. And all of this on top of a container scheduler that allows GitLab, the GitLab Runners for CI, and the applications that we deploy, to scale. Welcome to GitLab, the single application for the entire DevOps lifecycle, helping you bring modern applications from plan to monitor, quickly and reliably.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/demo/integrations.md

In some cases, GitLab must be used with existing systems. The most common systems requested include Atlassian Jira for issue management, Jenkins for pipeline execution or GitHub for source code management. Jira to GitLab workflow, GitHub to GitLab CI/CD linkage or GitLab to Jenkins connections can be arranged quickly on a per-project basis using available integrations from GitLab.

The below demonstration highlights a simple flow of work between Jira issues and GitLab source code management, as well as between GitLab merge requests and Jenkins pipelines.

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/Jn-fyra7xQ" frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

The below demonstration highlights a simple flow of work between GitHub pull requests and GitLab CI/CD.

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/qgl3F2j-1cI" frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

Jira Integration Demo

There are 3 different Jira integrations avaialble.

1. Real-time MR/comments integration also known as "Jira Integration". All Jira + GitLab customers should use this if they cannot use only GitLab.
2. Dev Panel (DVCS) integration. Only way to have GitLab feed Jira's Dev Panel if using GitLab self-managed and/or Jira Server. Data sync once per hour Premium
3. Dev Panel integration using the GitLab for Jira app from the Atlassian Marketplace. Ideal for Jira Cloud integrating with GitLab SaaS because data is sync'd in real-time! - Works only with Jira Cloud and GitLab SaaS Premium

The following guide can be used to integrate GitLab.com with Jira Software Cloud:

- GitLab Jira integration - mention a Jira issue ID from GitLab and have this reflected in the Jira Issue's comments.
- Dev Panel integration using the GitLab for Jira app - for each Jira issue, displays links with number of related commits, branches, and pull (merge) requests from GitLab.

Prerequisite: Must be a Premium group owner on GitLab.com

GitLab Jira integration

1. You can either create your own free Jira Software Cloud environment or use the environment listed under Jira Integration Demo Login in 1Password.
   - If using the Jira Integration Demo Login select the spring-integrations project in Jira and go to the issue board.
2. Create a new Jira issue. Note the ID or copy it to the clipboard.
3. Log in to GitLab.com and create a new project within your Ultimate group
   - Under Settings > Integrations > Jira, configure the integration with the web URL, username, API token (you will need to generate a new token for your own use), and transition ID for your project.
4. Create a branch from Repository > Branches. Include your new Jira issue ID in branch name and description (such as fixes-SI-X, where X is the issue number).
5. Create a merge request with SI-X in the name and Resolves SI-X in the description.
6. Edit any non-essential code within the repository, then enter a commit message mentioning SI-X again.
7. When you merge the GitLab Merge Request, the Jira issue's status is transitioned to Done.
8. Go to Jira again via the link in the GitLab menu.
9. Navigate to the Jira issue board and select your issue (SI-X if using the spring-integrations project). Note the GitLab content is now present in the Comments area. If you have Jira open in another browser tab, the updated comments will show immediately upon refresh of the page's content.

Dev Panel integration using the GitLab for Jira app

1. Follow the GitLab.com Development Panel instructions.
2. Navigate to the Jira issue board and select your issue (SI-X if using the spring-integrations project). Note the GitLab commit and branch information displayed in the Development panel on the right side.

Jenkins Integration Demo

- Login to Jenkins using Jenkins.Taunki.Cloud Login in 1Password.
- Log into GitLab spring-integrations project on demo.tanuki.cloud via 1Password.
- Create a branch from Repository>Branches.
- Create a merge request.
- Edit any non-essential code within the repository and enter a generic commit message.
- Go to the created merge request and click on the pipeline to bring up the graphical view.
- Note the pipeline actually ran a Jenkins job in an external stage.
- Click on Jenkins job, noting that it